

UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL
INSTITUTO DE INFORMÁTICA

JULIANA RODRIGUES

**Relatório de Análise Experimental: Seleção
do k -ésimo Elemento**

Porto Alegre

UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL

Reitora: Prof^ª. Marcia Cristina Bernardes Barbosa

Vice-Reitor: Prof. Pedro de Almeida Costa

Pró-Reitora de Pós-Graduação: Prof^ª. Claudia Wasserman

Diretor do Instituto de Informática: Prof. Luciano Paschoal Gaspar

:

Bibliotecário-chefe do Instituto de Informática: Alexsander Borges Ribeiro

1 RELATÓRIO DO EXPERIMENTO

1.1 Introdução

Este relatório descreve a avaliação empírica de três algoritmos para a **seleção do k -ésimo menor elemento** de um vetor não ordenado: a **Seleção Ingênua** (NaiveSelection: copia, ordena e indexa), o **Quickselect Aleatorizado** (RandomizedQuickSelect: pivô sorteado, partição, recursão) e a **Mediana das Medianas** (MedianOfMediansSelection: pivô determinístico via mediana das medianas de grupos de tamanho g , partição, recursão).

O objetivo central é determinar *qual algoritmo é preferível na prática* e, quando não há vencedor absoluto, identificar os pontos de cruzamento de desempenho. A análise verifica as complexidades assintóticas teóricas por contagem de operações, traduz essas assíntotas em constantes reais de tempo, caracteriza a distribuição estatística do tempo do Quickselect, investiga a dependência em relação ao posto k procurado, mede o efeito do tamanho de grupo g da mediana das medianas (incluindo a atenção especial requerida ao caso $g = 3$) e investiga o que a biblioteca padrão (`std::nth_element`) realmente implementa.

Todas as conclusões reportam o que os dados de fato mostram. Um achado central, discutido na Seção 1.4.7, é que a mediana das medianas com partição de Lomuto *não preserva* sua garantia linear sob entradas com muitas chaves repetidas, um resultado que motiva diretamente a investigação do `std::nth_element`.

1.2 Instâncias de Teste e Plano de Experimentos

1.2.1 Geradores de Instâncias

Cada instância é um vetor de inteiros gerado de forma determinística a partir da tripla (tamanho n , distribuição, semente), de modo que *a mesma instância é reproduzida identicamente* para todos os algoritmos comparados em uma célula, requisito de comparação justa. Apenas a aleatoriedade interna do Quickselect (semente do gerador de pivô) varia de forma independente. As nove distribuições implementadas estão na Tabela 1.1.

Tabela 1.1 – Catálogo de geradores de instâncias.

ID	Descrição	Estressa
uni	n valores distintos, permutação aleatória	caso médio (baseline)
gaussian	valores $\sim \mathcal{N}(0, 1)$ escalonados	realismo
alleq	todos os elementos iguais	partição de Lomuto
dup_2	apenas 2 valores distintos	baixa cardinalidade
dup_5	apenas 5 valores distintos	baixa cardinalidade
sorted_asc	já ordenado crescente	ordem adversária
sorted_desc	já ordenado decrescente	ordem adversária
organpipe	crescente depois decrescente (“tubo de órgão”)	armadilha de mediana-de-3
nearly_sorted	ordenado + 1% de trocas aleatórias	quase ordenado

O gerador `uni` produz uma *permutação aleatória* de $\{0, \dots, n - 1\}$ (valores distintos): como as contagens de comparação dependem apenas da ordem relativa, isso satisfaz a hipótese de chaves distintas usada no ajuste teórico do experimento E4. Uma distribuição desconhecida faz o gerador lançar exceção (em vez de substituir silenciosamente por uniforme), evitando rotular dados incorretamente.

1.2.2 Plano de Experimentos

Os experimentos são organizados em nove grupos (E1 a E9), cada um com objetivo, variável independente e métrica fixos (Tabela 1.2). Os experimentos de *contagem* (E1, E4, E7) são determinísticos quanto às operações e podem ser executados em paralelo; os de *tempo* (E2, E3, E5, E6) são executados sequencialmente em um único núcleo, com o binário compilado em modo *release* (sem *AddressSanitizer*), para não enviesar os tempos absolutos.

Tabela 1.2 – Resumo dos experimentos realizados. k medular = $\lfloor n/2 \rfloor$ salvo indicação contrária.

Teste	Objetivo	n / variação	Reps.
E1	Contagens (comparações, acessos) vs. n	10^2 a 5×10^5	10 sementes
E2	Tempo $T(n)$ + ajuste de regressão	5×10^3 a $2,5 \times 10^5$	30
E3	Distribuição de T do Quickselect (n fixo)	$10^3, 10^4, 10^5$	500
E4	Dependência do posto k	k/n varrido ($n=10^4, 10^5$)	5
E5	Varredura de tamanho de grupo g	$g \in \{3, 5, 7, 9, 11, 15, 21\}$	30
E6	Anomalia $g = 3$	10^3 a $10^6, g \in \{3, 5, 7\}$	8
E7	Robustez / entradas adversárias	9 distrib. $\times n$ até 4000	3
E8	Estudo de cruzamento prático	reusa E2	n/a
E9	Investigação do <code>std::nth_element</code>	reusa E7 + E2	n/a

1.2.3 Esquema do Arquivo CSV

Cada execução do binário `run_experiments` anexa uma linha em formato *longo* a `data/benchmark_data.csv` (uma linha por ensaio), e ao final uma cópia datada (`benchmark_data_AAAAMDDThhmmss.csv`) é arquivada. As colunas relevantes são:

```
experiment_id, algorithm, n, k, k_frac, g, distribution,
instance_seed,
comparisons, accesses, recursion_depth, time_ns, build,
oracle_match, timestamp
```

As colunas `accesses` e `recursion_depth` ficam vazias quando não fazem sentido (acessos não são instrumentáveis no `std::nth_element`; profundidade só é registrada para Quickselect e mediana das medianas). A coluna `oracle_match` registra a verificação de correção descrita adiante.

1.2.4 Protocolo de Medição e Salvaguardas

Correção (oráculo). Antes de registrar qualquer métrica, o resultado do algoritmo sob teste é comparado ao da `NaiveSelection` sobre a *mesma* instância. Em todas as 3 663 linhas geradas, `oracle_match` foi verdadeiro, nenhuma divergência.

Tempo. Mede-se com `std::chrono::steady_clock` (monotônico); reporta-se a *mediana* de múltiplas repetições, robusta a *jitter* de escalonamento. Como cada ensaio é um processo independente, o aquecimento de cache entre ensaios é irrelevante; a mediana cumpre o papel da medição robusta.

Salvaguarda de tempo limite. Cada célula tem um *timeout* de 20 s. Células degeneradas (Quickselect ou mediana das medianas sobre *alleq*) podem explodir para comportamento quadrático ou pior; ao estourar o limite, a célula simplesmente não gera linha, e a análise tolera a célula ausente.

1.2.5 Ambiente de Execução

Todos os experimentos foram executados na máquina descrita na Tabela 1.3.

Tabela 1.3 – Configuração do ambiente de execução.

Componente	Especificação
Processador	Intel Core Ultra 5 236V, 8 núcleos, 1 thread/núcleo, até 4,7 GHz
Cache L1	320 KiB dados + 512 KiB instruções (8 instâncias)
Cache L2	14 MiB (5 instâncias)
Cache L3	8 MiB (1 instância)
Memória RAM	16 GiB
Sistema operacional	Ubuntu 24.04.4 LTS, kernel 6.17.0-35-generic
Compilador	g++ 13.3.0 (Ubuntu 13.3.0-6ubuntu2~24.04.1)
Flags (release)	-std=c++17 -O3 -DNDEBUG

Os processos concorrentes do sistema operacional não foram controlados explicitamente; os tempos reportados correspondem à mediana de repetições independentes, atenuando interferências pontuais.

1.3 Algoritmos e Análise de Complexidade

1.3.1 Seleção Ingênua

A `NaiveSelection` copia o vetor (passagem por valor), ordena com `std::sort` e indexa a posição k :

Listing 1.1 – Núcleo de NaiveSelection.h

```

1 T select(std::vector<T> data, int k, MetricsContext& metrics) override {
2     std::sort(data.begin(), data.end(), [&metrics](const T& a, const T& b) {
3         metrics.comparisons++;
4         return a < b;
5     });
6     metrics.accesses++;
7     return data[k];
8 }

```

Complexidade: a cópia é $O(n)$, a ordenação introsort da biblioteca é $\Theta(n \log n)$ comparações e a indexação é $O(1)$. Total: $\Theta(n \log n)$, independente de k . Os acessos ao vetor não são instrumentados (ocorrem dentro do `std::sort`).

1.3.2 Quickselect Aleatorizado

O `RandomizedQuickSelect` sorteia um pivô uniformemente, particiona (esquema de Lomuto) e recorre apenas no lado que contém o posto k :

Listing 1.2 – Partição de Lomuto e recursão (`RandomizedQuickSelect.h`)

```

1 int partition(std::vector<T>& data, int left, int right, int pivotIndex,
2     MetricsContext& metrics) {
3     T pivotValue = data[pivotIndex];
4     metrics.accesses++;
5     std::swap(data[pivotIndex], data[right]); metrics.accesses += 4;
6     int storeIndex = left;
7     for (int i = left; i < right; i++) {
8         metrics.comparisons++; metrics.accesses++;
9         if (data[i] < pivotValue) {
10             std::swap(data[storeIndex], data[i]); metrics.accesses += 4;
11             storeIndex++;
12         }
13     }
14     std::swap(data[right], data[storeIndex]); metrics.accesses += 4;
15     return storeIndex;
16 }
17 // ... na recursao:
18 std::uniform_int_distribution<int> dist(left, right);
19 int pivotIndex = partition(data, left, right, dist(rng), metrics);
20 if (k == pivotIndex) return data[k];
21 else if (k < pivotIndex) return quickSelect(data, left, pivotIndex - 1, k, metrics,
    depth + 1);
22 else return quickSelect(data, pivotIndex + 1, right, k, metrics,
    depth + 1);

```

Complexidade: o pivô aleatório dá, em valor esperado, partições suficientemente

equilibradas, resultando em $O(n)$ comparações esperadas (a recorrência $T(n) = T(n/2) + O(n)$ no caso típico soma uma série geométrica $2n$). No *pior caso* (pivôs sistematicamente ruins, ou todos os elementos iguais, ver E7), degrada para $\Theta(n^2)$. A partição de Lomuto envia os elementos *iguais* ao pivô para o mesmo lado, o que é a raiz da degradação em *alleq*.

1.3.3 Mediana das Medianas

A `MedianOfMediansSelection` escolhe um pivô determinístico: divide o segmento em grupos de tamanho g , ordena cada grupo para obter sua mediana, e recorre sobre o vetor de medianas para achar a mediana das medianas:

Listing 1.3 – Seleção do pivô determinístico (MedianOfMedians.h)

```

1 T get_pivot(std::vector<T>& data, int left, int right, MetricsContext& metrics, int
    depth = 1) {
2     int n = right - left + 1;
3     std::vector<T> medians;
4     for (int i = 0; i < n; i += g) {
5         int subRight = std::min(left + i + g - 1, right);
6         medians.push_back(get_median_of_small_group(data, left + i, subRight, metrics));
7     }
8     return momSelect(medians, 0, medians.size() - 1, medians.size() / 2, metrics, depth
        + 1);
9 }

```

Em seguida, particiona em torno do pivô (também por Lomuto) e recorre no lado relevante.

Complexidade (teórica): para g ímpar, a mediana das medianas garante que pelo menos $\approx n(g+1)/(4g)$ elementos ficam de cada lado, logo o lado maior tem no máximo $\approx n(3g-1)/(4g)$ elementos; o subproblema das medianas tem n/g elementos. A recorrência $T(n) \leq T(n/g) + T(\frac{3g-1}{4g}n) + O(n)$ *contrai* (é linear) se e somente se $\frac{1}{g} + \frac{3g-1}{4g} < 1$, i.e. $g \geq 5$. Para $g = 3$ a soma é exatamente 1, sem garantia de linearidade (ver E6). Assim, no papel, a mediana das medianas é $\Theta(n)$ de pior caso para $g \geq 5$.

Ressalva de implementação (importante): essa garantia linear pressupõe partição que trate corretamente elementos *iguais* ao pivô. A partição de Lomuto de duas vias usada aqui não o faz: sob *alleq*, cada nível elimina apenas um elemento, e a recursão aninhada de `get_pivot` faz o custo explodir para muito além de $\Theta(n^2)$ (Seção 1.4.7). Optou-se por *reportar* esse comportamento como achado, em vez de mascará-lo com uma partição de três vias.

1.3.4 Biblioteca Padrão: `std::nth_element`

A `std::nth_element` (libstdc++) implementa **introsselect**: um Quickselect com pivô por mediana-de-3 que monitora a profundidade de recursão e, ao ultrapassar um limiar $\approx 2 \log_2 n$, recua para um método de pior caso garantido (*heapselect*). Assim evita simultaneamente o $\Theta(n^2)$ do Quickselect puro e o alto fator constante da mediana das medianas no caso comum. As comparações são instrumentáveis via comparador customizado (um *lambda* que incrementa o contador); os acessos/trocas internos não o são, e isso é documentado como limitação.

1.3.5 Resumo Teórico

Tabela 1.4 – Complexidades teóricas (comparações) dos algoritmos.

Algoritmo	Complexidade	Observação
Seleção Ingênua	$\Theta(n \log n)$	independente de k
Quickselect Aleatorizado	$O(n)$ esperado / $\Theta(n^2)$ pior	pior caso em chaves iguais
Mediana das Medianas ($g \geq 5$)	$\Theta(n)$ pior caso*	*exige partição que trate duplicatas
Mediana das Medianas ($g = 3$)	sem garantia linear	$\frac{1}{g} + \frac{3g-1}{4g} = 1$
<code>std::nth_element</code> (introsselect)	$O(n)$ pior caso	Quickselect + recuo p/ heapselect

1.4 Resultados Experimentais e Análise

1.4.1 E1: Contagens de Operações vs. n

O teste E1 mede comparações e acessos ao vetor em função de n (instâncias `uni`, k medular, $g = 5$, média de 10 sementes). A Tabela 1.5 traz os expoentes de crescimento ajustados por regressão log-log, e a Figura 1.1 mostra as curvas de comparações.

Tabela 1.5 – Expoentes empíricos de crescimento (E1, ajuste k em n^k).

Algoritmo	Métrica	k_{emp}	R^2
Seleção Ingênua	comparações	1,123	0,9997
Quickselect	comparações	0,995	0,9968
Mediana das Medianas	comparações	1,038	0,9996
Quickselect	acessos	0,984	0,9975
Mediana das Medianas	acessos	1,030	0,9998
Quickselect	acessos/comparações (razão)	3,10	
Mediana das Medianas	acessos/comparações (razão)	1,14	

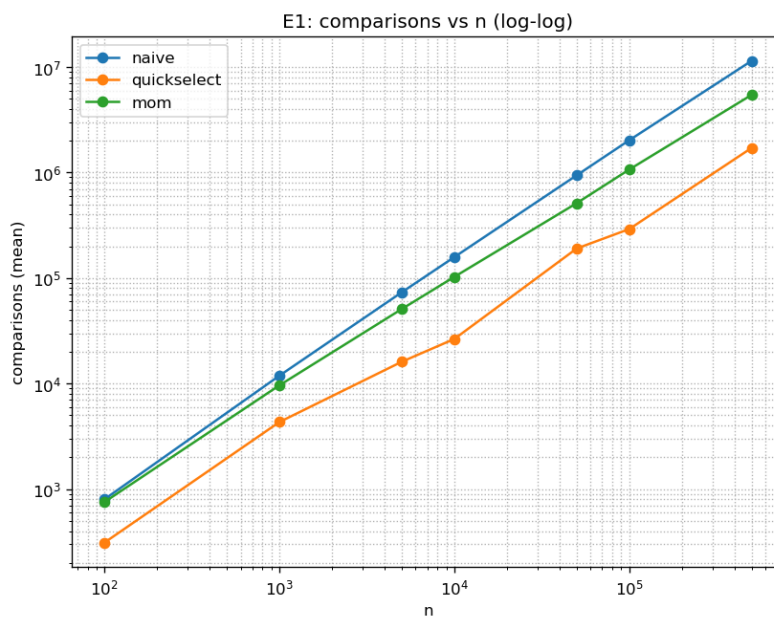


Figura 1.1 – Comparações médias vs. n em escala log-log (E1). A Seleção Ingênua ($k_{\text{emp}} = 1,12$) cresce como $n \log n$; Quickselect (0,99) e Mediana das Medianas (1,04) são essencialmente lineares.

Os expoentes confirmam as classes teóricas: a Seleção Ingênua tem expoente 1,12 (acima de 1, refletindo o fator $\log n$), enquanto Quickselect (0,99) e Mediana das Medianas (1,04) são lineares. Em números absolutos, a $n = 5 \times 10^5$ a Seleção Ingênua faz $\approx 11,4$ milhões de comparações, contra 5,4 milhões da Mediana das Medianas e apenas 1,7 milhão do Quickselect. A razão acessos/comparações $\approx 3,1$ do Quickselect é a assinatura da partição de Lomuto (cada troca custa 4 acessos extras), coerente com a faixa 3 a 4 esperada; a Mediana das Medianas, que faz menos trocas relativas, tem razão $\approx 1,1$.

1.4.2 E2: Tempo de Parede e Ajuste de Regressão

O teste E2 mede $T(n)$ no *release* e ajusta três modelos candidatos ($an+b$, $a n \ln n+b$, $an^2 + b$) por mínimos quadrados, selecionando o de maior R^2 (Tabela 1.6). A Figura 1.2 sobrepõe os dados e as curvas ajustadas.

Tabela 1.6 – Modelos de tempo: melhor ajuste por algoritmo (E2). a em ns/unidade do modelo.

Algoritmo	Melhor modelo	a	b (ns)	R^2
Seleção Ingênua	$a n \ln n + b$	4,71	$-1,12 \times 10^5$	0,9980
Quickselect	$a n + b$	9,67	$-1,03 \times 10^4$	0,9988
Mediana das Medianas	$a n + b$	25,65	$+1,48 \times 10^5$	0,9954
<code>std::nth_element</code>	$a n + b$	8,70	$+1,5 \times 10^2$	0,9979

Tabela 1.7 – Tempos medianos (μ s) vs. n (E2, 30 repetições).

n	Ingênua	Quickselect	Med. Medianas	<code>nth_element</code>
5 000	208,6	36,6	131,4	33,4
10 000	453,6	80,6	268,1	86,6
25 000	1 238,5	189,0	792,2	171,7
50 000	2 305,4	526,4	1 716,0	452,8
100 000	4 865,6	962,1	2 781,2	932,4
250 000	14 704,5	2 400,3	6 484,7	2 150,5

O modelo $n \ln n$ vence para a Seleção Ingênua ($R^2 = 0,9980$ contra 0,9949 do linear), enquanto Quickselect, Mediana das Medianas e `nth_element` são melhor descritos por modelos *lineares*. As constantes empíricas são o que de fato decide o vencedor prático dentro de uma mesma classe: 9,7 ns/elemento do Quickselect e 8,7 ns/elemento do `nth_element` contra 25,7 ns/elemento da Mediana das Medianas, ou seja, a Mediana das Medianas é linear, mas com fator constante $\approx 2,7\times$ maior, fruto do custo de ordenar cada grupo e da recursão aninhada de medianas.

1.4.3 E3: Distribuição do Tempo do Quickselect

O teste E3 caracteriza T (e as comparações) do Quickselect como variável aleatória, com $M = 500$ execuções independentes para $n \in \{10^3, 10^4, 10^5\}$. A Tabela 1.8 resume as

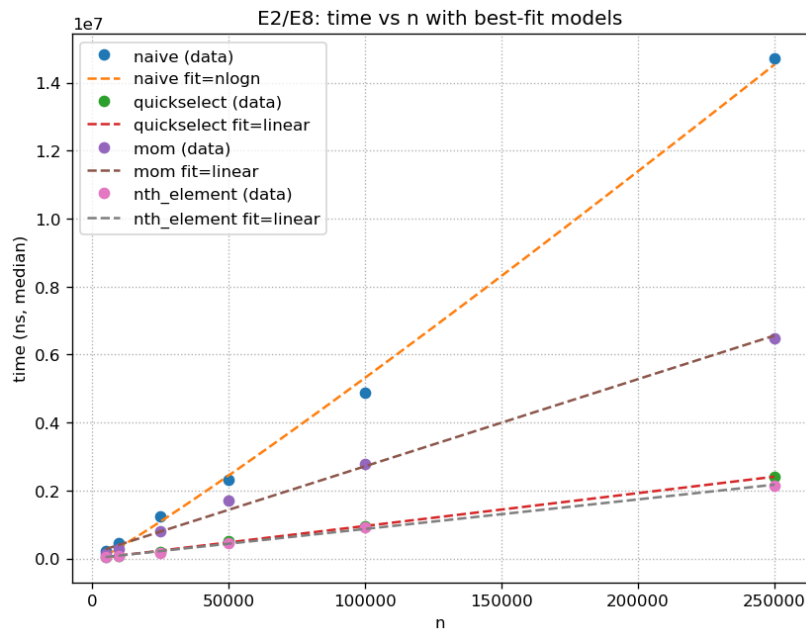


Figura 1.2 – Tempo mediano vs. n com os melhores modelos ajustados (E2). A Seleção Ingênua é a única bem ajustada por $n \ln n$; as demais são lineares.

estatísticas e a Figura 1.3 mostra os histogramas de comparações.

Tabela 1.8 – Estatísticas da distribuição de comparações do Quickselect (E3, $M = 500$).

n	Média	Variância	Desvio	CV	Assimetria
1 000	3 326	$9,7 \times 10^5$	986	0,296	0,62
10 000	34 031	$9,9 \times 10^7$	9 961	0,293	0,83
100 000	334 120	$8,4 \times 10^9$	91 448	0,274	0,62

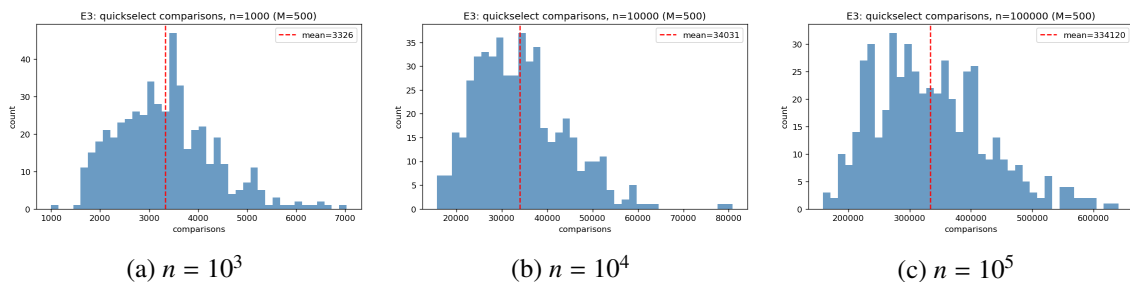


Figura 1.3 – Histogramas de comparações do Quickselect (E3). A cauda à direita (assimetria positiva $\approx 0,6$ a $0,8$) reflete sequências ocasionais de pivôs infelizes; nenhum *outlier* foi descartado.

A média escala como $n^{1,00}$ e o desvio-padrão como $n^{0,98}$, logo a **variância escala como $n^{1,97} \approx n^2$** , exatamente a previsão teórica de que média e desvio crescem ambos linearmente em n . Como consequência, o coeficiente de variação $CV = \sigma/\mu$ permanece *praticamente constante* ($\approx 0,27$ a $0,30$): a variabilidade relativa não cresce com a escala.

As distribuições são moderadamente assimétricas à direita (assimetria 0,6 a 0,8), confirmando a presença de uma cauda de pivôs infelizes que, contudo, não se agrava em termos relativos com n .

1.4.4 E4: Dependência do Posto k

O teste E4 varre k/n para $n \in \{10^4, 10^5\}$ (Figura 1.4, Tabela 1.9). A Seleção Ingênua é, por construção, *independente de k* (ordena tudo): 2,05 milhões de comparações para qualquer k a $n = 10^5$, um teste de sanidade que confirma o custo de ordenação total. A Mediana das Medianas é quase plana ($\approx 1,07$ milhão), graças à sua divisão garantida $\approx 30/70$ independentemente de k . O Quickselect mostra dependência *moderada*: mais barato sobretudo no extremo superior (≈ 182 mil em $k/n = 0,99$) e com um pico em postos interiores (≈ 322 mil em $k/n = 0,75$). A forma é qualitativamente compatível com a curva esperada $2[n + k \ln(n/k) + (n - k) \ln(n/(n - k))]$, mas o efeito é modesto frente ao ruído e o pico empírico não cai exatamente na mediana prevista pela fórmula simétrica, ou seja, a corcova teórica é apenas *sugerida*, não confirmada com nitidez.

Tabela 1.9 – Comparações médias vs. posto k/n em $n = 10^5$ (E4).

k/n	Ingênua	Quickselect	Med. Medianas
0,00	2 047 733	242 418	1 068 956
0,25	2 047 733	289 708	1 076 239
0,50	2 047 733	252 335	1 069 369
0,75	2 047 733	322 211	1 064 423
0,99	2 047 733	184 203	1 068 170

1.4.5 E5: Tamanho de Grupo da Mediana das Medianas

O teste E5 varia g (instâncias uni, $n = 10^5$, k medular) e analisa o efeito *teórico e empírico*. A Tabela 1.10 reúne ambas as facetas: as métricas medidas (comparações, tempo, profundidade de recursão) e a teoria de poda por grupo.

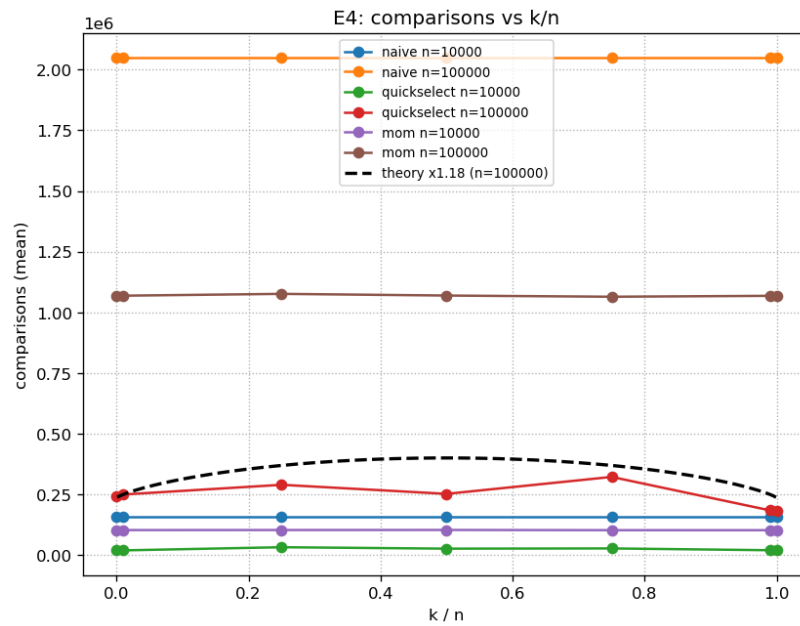


Figura 1.4 – Comparações vs. k/n (E4). Ingênua plana (independente de k); Mediana das Medianas quase plana; Quickselect com dependência moderada, mais barato nos extremos. A curva tracejada é o ajuste teórico do Quickselect.

Tabela 1.10 – Efeito do tamanho de grupo g (E5): empírico (comparações, tempo mediano, profundidade) e teórico (fração do lado maior $\frac{3g-1}{4g}$ e soma da recorrência).

g	Comparações	Tempo (μ s)	Prof. rec.	Frac. recursão	Linear?
3	1 420 506	5 567,8	16,4	0,667	não (soma = 1,00)
5	1 080 606	2 728,9	15,6	0,700	sim (0,90)
7	1 053 384	2 397,8	15,0	0,714	sim (0,86)
9	1 120 793	2 229,2	14,7	0,722	sim (0,83)
11	1 177 582	2 727,5	14,6	0,727	sim (0,82)
15	1 325 278	2 495,4	14,3	0,733	sim (0,80)
21	1 483 573	2 835,2	13,7	0,738	sim (0,79)

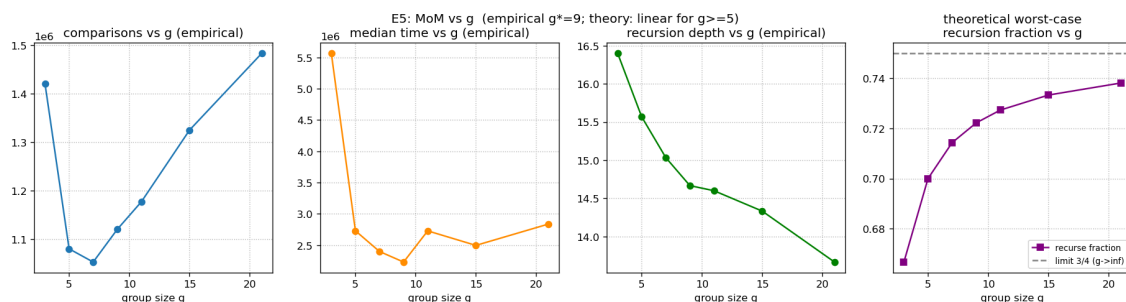


Figura 1.5 – Mediana das Medianas vs. g (E5): comparações, tempo e profundidade de recursão (empíricos) e a fração teórica de recursão $\frac{3g-1}{4g}$ (que tende a $3/4$). A curva de comparações tem forma de U.

O comportamento empírico tem a forma de **U** prevista: g pequeno gera muitos níveis de recursão (mais comparações), enquanto g grande encarece a mediana por grupo (a ordenação de cada grupo cresce com g). As comparações são minimizadas em $g = 7$ (1,05 milhão) e o tempo mediano em $g = 9$ (2,23 ms); o $g = 5$ comumente citado é competitivo, mas não ótimo nesta máquina. A profundidade de recursão cai monotonicamente com g (de 16,4 em $g = 3$ a 13,7 em $g = 21$): mais grupos por nível significam menos níveis, o que fornece a explicação mecanística da forma das outras curvas. A coluna teórica mostra a transição-chave: $g = 3$ tem soma de recorrência exatamente 1,00 (sem garantia linear), e todo $g \geq 5$ contrai.

1.4.6 E6: A Anomalia $g = 3$ (Atenção Especial)

O teste E6 investiga por que $g = 3$ não é provadamente linear. A recorrência $T(n) \leq T(n/3) + T(\frac{3 \cdot 3 - 1}{4 \cdot 3}n) + O(n) = T(n/3) + T(2n/3) + O(n)$ tem soma de frações $\frac{1}{3} + \frac{2}{3} = 1$: ela *não contrai*, podendo derivar para $\Theta(n \log n)$ ou pior no *pior caso*. Para quantificar empiricamente, varremos n de 10^3 a 10^6 com $g \in \{3, 5, 7\}$ sobre **uni** e **sorted_asc** (Tabela 1.11, Figura 1.6).

Tabela 1.11 – Expoentes empíricos de comparações vs. n por g (E6, até $n = 10^6$).

Distribuição	g	k_{emp}	R^2
uni	3	1,038	0,9999
uni	5	1,016	0,9999
uni	7	1,016	0,9999
sorted_asc	3	1,014	0,980
sorted_asc	5	1,015	1,000
sorted_asc	7	1,070	0,989

Em entradas uniformes e ordenadas, $g = 3$ permanece linear mesmo a $n = 10^6$ ($k_{\text{emp}} = 1,04$ em **uni**, indistinguível dos 1,02 de $g = 5/7$). A degradação de $g = 3$ é um fenômeno de *pior caso*: a recorrência não contrair é uma cota estrutural, mas em dados benignos a mediana das medianas escolhe pivôs muito melhores que a garantia $\frac{1}{3}$, produzindo divisões quase equilibradas e, portanto, tempo linear. A profundidade de recursão de $g = 3$ é apenas ligeiramente superior à de $g \geq 5$ (16,4 vs. 15,6 em $n = 10^5$, Tabela 1.10). A recomendação prática que decorre é a mesma adotada pelo introslect:

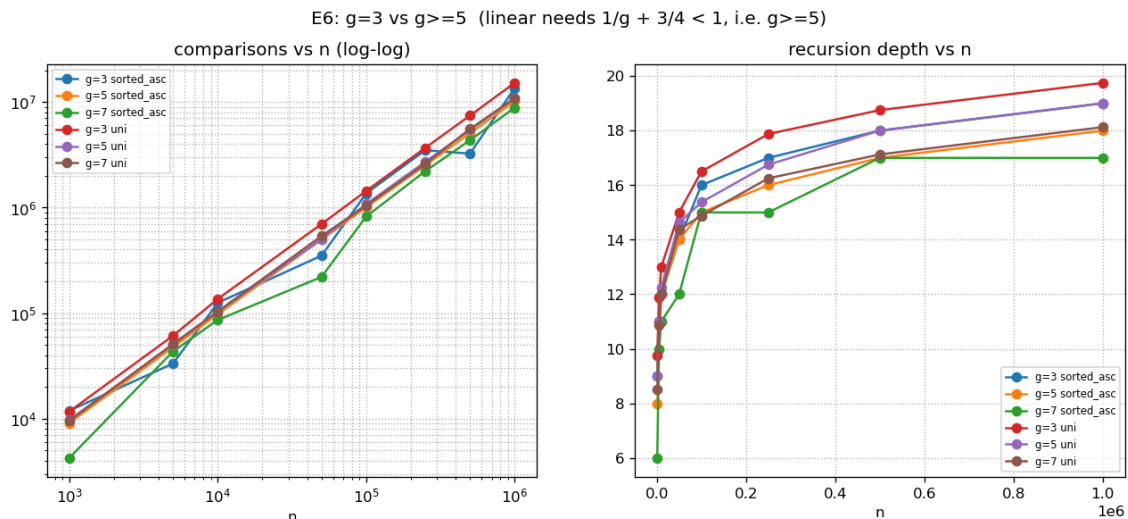


Figura 1.6 – Anomalia $g = 3$ (E6): comparações vs. n (log-log) e profundidade de recursão vs. n , para $g \in \{3, 5, 7\}$. Em entradas uni/sorted, $g = 3$ não se separa visivelmente de $g \geq 5$ mesmo a $n = 10^6$.

usar $g \geq 5$ (ou um recuo híbrido) para *garantir* a linearidade, já que a ausência de garantia em $g = 3$ poderia ser explorada por uma entrada adversária construída para tanto.

1.4.7 E7: Robustez e Estresse com Duplicatas

O teste E7 submete os algoritmos às famílias adversárias e de baixa cardinalidade (Tabela 1.12, Figuras 1.7 e 1.8).

Tabela 1.12 – Comparações médias por distribuição em $n = 4000$ (E7). Em negrito, as explosões super-lineares.

Distribuição	Ingênua	Quickselect	Med. Medianas	nth_element
uni	57 280	11 649	34 007	11 810
gaussian	57 374	12 343	41 034	10 628
sorted_asc	54 413	11 548	38 423	10 018
organpipe	113 036	17 038	33 742	42 203
dup_5	40 666	251 388	21 041 285	10 306
dup_2	40 562	745 884	324 989 362	10 748
alleq	36 939	6 000 999	<i>timeout</i>	8 035

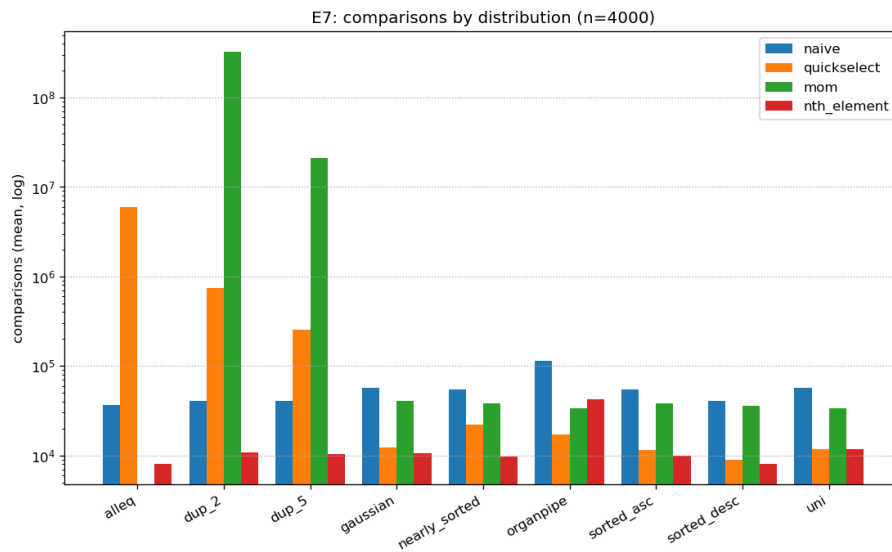


Figura 1.7 – Comparações por distribuição em escala log (E7, $n = 4000$). Quickselect e Mediana das Medianas explodem em entradas de baixa cardinalidade; Ingênua e `nth_element` permanecem controlados.

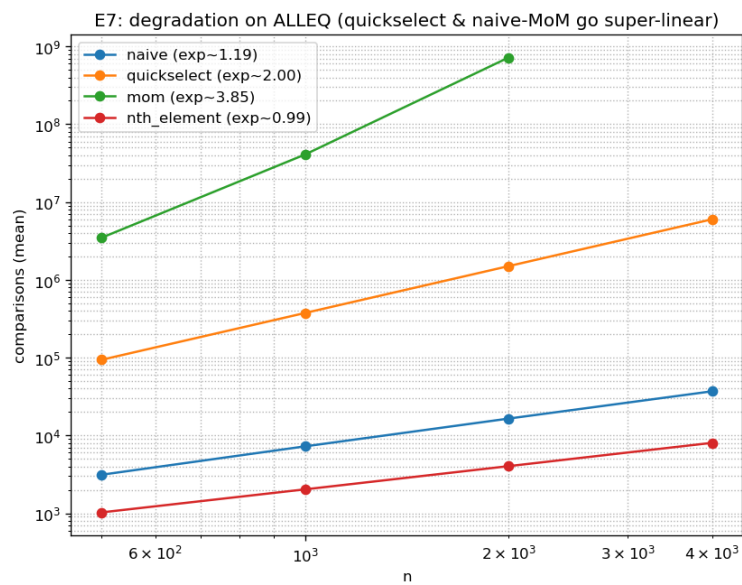


Figura 1.8 – Degradação em alleq: comparações vs. n (log-log). Os expoentes ajustados revelam a natureza de cada falha (Tabela 1.13).

Tabela 1.13 – Expoente de comparações vs. n na entrada `alleq` (E7).

Algoritmo	k_{emp} (<code>alleq</code>)	Diagnóstico
Seleção Ingênua	1,19	$n \log n$ (sem degradação)
<code>std::nth_element</code>	0,99	linear (introselct resiste)
Quickselect	2,00	quadrático (Lomuto + chaves iguais)
Mediana das Medianas	3,85	catastrófico (recursão aninhada)

Sob `alleq`, o Quickselect degrada para *exatamente* $\Theta(n^2)$ (expoente 2,00): com todos os elementos iguais, a partição de Lomuto envia tudo para um lado e cada nível remove apenas um elemento. O mesmo mecanismo, agravado pela recursão aninhada de `get_pivot`, leva a Mediana das Medianas a um expoente 3,85, pior que cúbico; a célula a $n = 4000$ sequer terminou dentro do *timeout* de 20 s. **A mediana das medianas com partição de Lomuto não preserva a garantia linear sob chaves repetidas.** A garantia de pior caso da literatura pressupõe uma partição que isole os elementos iguais ao pivô (partição de três vias); a implementação de duas vias aqui usada não o faz. As distribuições de *ordem* adversária (`sorted`, `organpipe`) não derrubam o Quickselect, pois seu pivô é *aleatório*, imune a arranjos fixos.

1.4.8 E8: Estudo de Cruzamento Prático

O teste E8 reusa os tempos de E2 (instâncias `uni`, k modular) para responder qual algoritmo vence na prática. A Tabela 1.14 lista o mais rápido por n .

Tabela 1.14 – Algoritmo mais rápido por n (E8, tempo mediano).

n	Mais rápido	Tempo (μs)
5 000	<code>nth_element</code>	33,4
10 000	<code>quickselect</code>	80,6
25 000	<code>nth_element</code>	171,7
50 000	<code>nth_element</code>	452,8
100 000	<code>nth_element</code>	932,4
250 000	<code>nth_element</code>	2 150,5

Sob entrada uniforme *não há um cruzamento limpo*: o Quickselect e o `std::nth_element` são estatisticamente **empatados** (ambos lineares, com constantes 9,7 e 8,7 ns/elemento),

alternando a liderança dentro do ruído de medição, o “cruzamento” detectado entre $n = 5000$ e 25000 é artefato de *jitter*, não um regime real. Ambos dominam a Mediana das Medianas (linear, mas $\approx 2,7\times$ mais lenta) e a Seleção Ingênua ($n \log n$, a mais lenta em toda a faixa) em todos os n . O ordenamento prático é, portanto: $\text{nth_element} \approx \text{Quickselect} < \text{Mediana das Medianas} < \text{Ingênua}$. Conclui-se que, para entrada aleatória, o *Quickselect* domina a partir de n pequeno e a *Mediana das Medianas* só é preferível sob as condições adversárias caracterizadas em E7, não por n isoladamente.

1.4.9 E9: Investigação do `std::nth_element` (Introselect)

O teste E9 confronta o `std::nth_element` aos três algoritmos sobre toda a suíte de E7 (Tabela 1.12, Figura 1.9). O resultado é a síntese de todo o relatório.

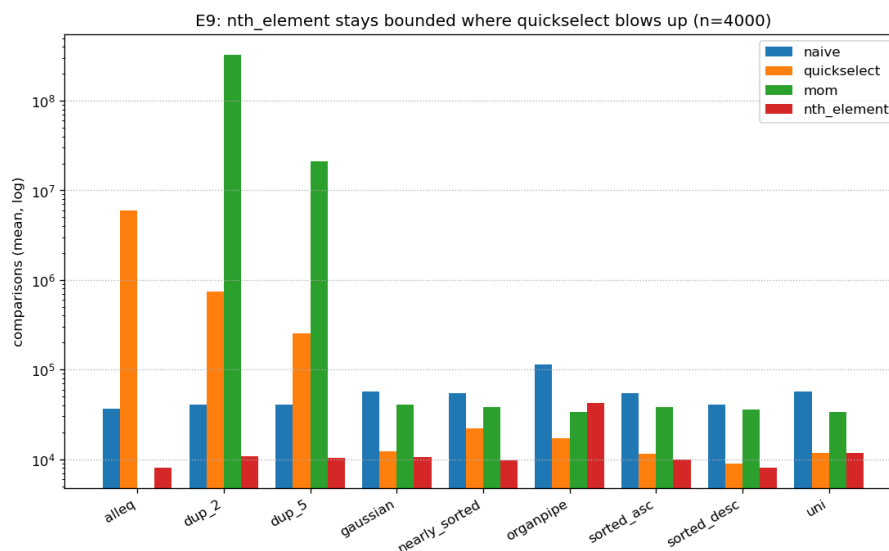


Figura 1.9 – `std::nth_element` vs. os três algoritmos na suíte adversária (E9, $n = 4000$, escala log). O `nth_element` mantém custo limitado onde o Quickselect e a Mediana das Medianas explodem.

O `std::nth_element` permanece **limitado em ≈ 8 a 12 mil comparações (≈ 2 a $3n$) em todas as distribuições**, incluindo `alleq` (8,0 mil) e `dup_2` (10,7 mil), enquanto o Quickselect explode para 6 milhões em `alleq` e a Mediana das Medianas para 325 milhões em `dup_2`. O `introselect` combina a velocidade média do Quickselect em `uni` (E2: praticamente empatados) com uma robustez que *nenhum* dos dois algoritmos puros possui, evita tanto o $\Theta(n^2)$ do Quickselect quanto a degradação por duplicatas da nossa mediana das medianas. Conclui-se que é por isso que a biblioteca padrão não usa nenhum dos três isoladamente: ela usa o Quickselect no caso comum, mas com mediana-de-3

e um *recuo* para um método de pior caso garantido quando a profundidade de recursão denuncia uma sequência de pivôs ruins.

1.5 Conclusão

Os experimentos verificaram as complexidades teóricas e, mais importante, expuseram onde a teoria de livro-texto encontra ressalvas de implementação:

Contagens e tempo (E1, E2). A Seleção Ingênua é $\Theta(n \log n)$ (expoente 1,12); Quickselect e Mediana das Medianas são lineares (expoentes 0,99 e 1,04). Em constantes de tempo, Quickselect (9,7 ns/elem.) e `nth_element` (8,7) batem a Mediana das Medianas (25,7) por fator $\approx 2,7\times$, apesar da mesma classe assintótica.

Distribuição e posto (E3, E4). O tempo do Quickselect tem média e desvio lineares em n (variância $\sim n^2$, CV $\approx 0,28$ constante) com cauda direita suave. A Seleção Ingênua é independente de k ; a Mediana das Medianas é quase plana; o Quickselect tem dependência moderada, mais barato nos extremos.

Tamanho de grupo e $g = 3$ (E5, E6). A curva de custo vs. g tem forma de U, com ótimo empírico em $g = 7$ (comparações) a $g = 9$ (tempo). A teoria de poda ($\frac{1}{g} + \frac{3g-1}{4g} < 1 \Leftrightarrow g \geq 5$) coincide com a ausência de garantia em $g = 3$; empiricamente, porém, $g = 3$ permanece linear em entradas benignas até $n = 10^6$, a anomalia é estritamente de pior caso.

Robustez e biblioteca padrão (E7, E9). O achado mais forte: sob *alleq*/baixa cardinalidade, o Quickselect degrada para $\Theta(n^2)$ e a *mediana das medianas com partição de Lomuto degrada ainda mais* (expoente 3,85), pois a partição de duas vias não isola chaves iguais. O `std::nth_element` (*introsselect*) é o único a permanecer linear em toda a suíte, justificando empiricamente por que as bibliotecas reais não adotam nenhum dos três algoritmos puros.

Recomendação prática (E8). Para entrada aleatória, *não há vencedor absoluto entre Quickselect e `std::nth_element`*, ambos lineares e empatados, dominando a Mediana das Medianas e a Seleção Ingênua em toda a faixa. A Seleção Ingênua só se justifica quando o vetor ordenado também é necessário; a Mediana das Medianas, como garantia de pior caso *desde que* sua partição trate duplicatas. Na ausência dessa correção, e no caso geral, o `std::nth_element` é a escolha recomendada: tão rápido quanto o Quickselect no caso médio e imune às patologias que derrubam os algoritmos puros.